DATA*6700 Final Project Report

A Pre-Trained Deep Learning Model for LiDAR Point Cloud Classification

Jason Ives - 1383642

Advisor: Dr. John Lindsay
Secondary Grader: Dr. Ben DeVries

University of Guelph
12 August 2026

Contents

Abstract

A pre-trained deep learning LiDAR point cloud classifier would be a valuable addition to the *Whitebox Next Gen* toolkit; however, such a tool must meet strict architectural, computational, and classification requirements. To address this, an end-to-end modeling pipeline was developed around a pure Rust implementation of PointNet, a powerful and efficient multi-layer perceptron-based classification architecture. The core PointNet model is augmented with high-performance pre- and post-processing steps, including eigenvalue feature derivation, class-weighted cross-entropy loss, and a tunable block overlap system. The pipeline integrates utilities for data preprocessing, training, metrics tracking, and classification, combining GPU-accelerated training with a platform-agnostic inference engine capable of classifying millions of points in minutes on standard hardware. This bifurcated pipeline maintains alignment with the "fast first" design philosophy of Whitebox by leveraging existing high-quality components, from the robust LiDAR processing tools within *Whitebox Next Gen* to *Burn*, a leading Rust machine learning framework. By strategically integrating these tools, the project delivers a forward-facing classification utility that is lightweight and free of external runtime dependencies, while maintaining a powerful and performant training system. The resulting classification model, pre-trained using data sampled from the high-quality ASPRS LAS 1.4-compliant IGN HD dataset, achieves an overall test accuracy of **84.70%** and a Mean Intersection over Union (MIoU) of **0.65** across all classes. This model does not represent a static end-point; rather, the underlying training and deployment framework enables the ongoing creation of improved or purpose-built LiDAR classifiers, reinforcing the flexibility and depth of the *Whitebox Next Gen* ecosystem.

1 Introduction

Developed in 1961 shortly after the laser's invention (McManamon, 2019), Light Detection and Ranging (LiDAR) has become a cornerstone technology for spatial data across environmental modeling, surveying, urban planning, and autonomous systems (Waykar, 2022). Standard LiDAR

systems calculate distances by measuring the time-of-flight (TOF) and return intensity of emitted laser pulses reflected off surfaces or atmospheric particles, building detailed 3D spatial representations over time.

These systems operate across multiple platforms and scales. Terrestrial LiDAR handles localized, fixed-scope modeling or vehicle-based dynamic state detection for self-driving cars and robotics. Overhead systems mounted on aircraft, drones, or satellites capture broad terrain characteristics and enable global monitoring.

As Waykar (2022) highlights, applications continue to expand rapidly. LiDAR data underpins Digital Elevation Models (DEMs) essential for cartography, infrastructure planning, and environmental tracking, while providing real-time spatial perception for robots and autonomous vehicles.

Most of these applications rely on secondary processing of the LiDAR dataset to extract a more meaningful representation. One of the most important and widely-used ways of processing LiDAR data is classification and semantic segmentation. LiDAR classification focuses on generating a single, global classification for the LiDAR point cloud. It is more often applied in terrestrial or smaller scope classification tasks where determining the primary element (a book, a tree, a person) in a scene is the goal. By contrast, semantic segmentation is focused on determining a classification for each point within the LiDAR point cloud. This is often applied to more broadly scoped LiDAR data sets that would be poorly represented by a single global classification.

LiDAR point cloud classification and semantic segmentation approaches fall into two broad categories, heuristic-based and deep learning-based. Heuristic-based methods generally work by analyzing the point neighbourhoods, and then implementing a set of deterministic decisions to filter and separate the points into different classes (Awrangjeb and Fraser, 2013). Soilán et al. (2020) note that this approach can be quite powerful, but relies on rigid, manually tuned thresholds which may not generalize well across diverse data sets. By contrast, deep learning-based approaches are known for exploiting complex geometric relationships in the dataset to achieve a higher performance ceiling, but are highly data dependent. Not only do deep learning-based approaches rely on a large amounts of training data to achieve their high performance, but unanticipated gaps in the training

data can leave them open to performance degradation in unseen situations.

Whitebox Next Gen (Whitebox Geospatial Inc., 2026) offers a powerful suite of heuristic-based LiDAR classification tools, including `classify_lidar` for identifying ground, building, electrical wire, and otherwise unclassified points, `classify_buildings_in_lidar` for refining and isolating building-specific points, and `filter_lidar_noise` for identifying and removing points classified as low and high noise. However, the *Whitebox Next Gen* ecosystem does not, to this point, have a deep learning-based LiDAR point cloud semantic segmentation utility. A properly executed deep learning semantic segmentation tool would be a valuable addition to the *Whitebox Next Gen* suite, joining the over 700 geospatial tools already available to users. However, any tool being considered for addition to the *Whitebox Next Gen* toolbox needs to adhere to a strict set of development principles:

- **Pure Rust:** No language bindings or wrapped non-Rust dependencies.
- **Minimal dependencies:** Prefers existing *Whitebox Next Gen* dependencies; any new additions must be lightweight and performant.
- **Cross-platform:** Fully agnostic across all major operating systems.
- **Standard hardware:** Runs on standard CPUs without requiring specialized hardware (e.g., GPUs).

With these design principles serving as the central tenets of the project, a pre-trained LiDAR point cloud semantic segmentation system for use within the *Whitebox Next Gen* ecosystem was developed, and is available now on GitHub (`https://github.com/JasonIves/lidar_point_cloud_classifier`). The project involved an architectural assessment and exploratory testing phase, an iterative Rust development cycle, and a testing and analysis phase. This workflow resulted in key deliverables: the `wb_lidar_train` system for model development and assessment, the `wb_lidar_classify` system for efficient user inference, and a pre-trained model based on the IGN HD (Institut national de l'information géographique et forestière (IGN), 2021) dataset. The

model training system includes a tunable data pre-processing pipeline, a data splitting mechanism designed to mitigate some of the known issues with splitting LiDAR point cloud data, a training mechanism that adapts to available processing hardware while maintaining important training metrics, and an evaluation system for testing the models on unseen data. The classification system includes a powerful preprocessing pipeline modeled on the training preprocessor, and a classification mechanism that quickly and efficiently classifies LiDAR point cloud files of any reasonable size using a user-selectable model, without relying on specialized hardware like GPUs. The pre-trained model provides a strong baseline model for general semantic segmentation, capturing key ASPRS LAS classes like ground (Class 2), building (Class 6), and different classes of vegetation (Classes 3, 4, and 5).

2 Methods

The process of developing the pre-trained LiDAR point cloud semantic segmentation pipeline involved several sequential steps designed to build the knowledge and tools necessary for creating and deploying the final pre-trained model. Assessments of candidate architectures, training and evaluation datasets, and the *Whitebox Next Gen* ecosystem took place in parallel, with the goal of better understanding the challenges and recent innovations in the field of LiDAR point cloud processing and how such a system would fit within the *Whitebox Next Gen* toolkit. This naturally led into an exploratory testing phase that was used to test a candidate architecture, and further explore what creating a data-to-model pipeline would entail. When the architectural details were confirmed, iterative AI-assisted development and testing began, a process that required a separate workflow definition and processing pipeline. As the modeling pipeline took shape and began to produce functional models, the work transitioned into a model tuning and assessment phase designed to ensure the final pre-trained model would be not only functional, but generalizable and performative.

2.1 Architecture Assessment

A number of different deep learning semantic segmentation architectures were considered. The convolutional neural network (CNN) is a reasonable starting point, since it offers proven and powerful classification capacity for pixel-based images. However, some key differences exist between LiDAR point clouds and pixel-based images which make the use of a standard CNN difficult (Sarker et al., 2024). LiDAR point clouds are fundamentally unordered and unstructured, and it is critical that the deep learning processing method should not treat them as ordered or structured data. CNNs rely on specific pixel spacing, locations, and relationships to accurately determine their kernel-based features - relationships that do not hold within LiDAR point cloud data. As a matter of course LiDAR point clouds have varying point density and spacing, making kernel construction challenging and poorly capturing the geometric features of LiDAR data.

To resolve some of these shortcomings, LiDAR point cloud-specific techniques have been developed. One branch of LiDAR point cloud classification techniques that stands out as particularly fitting when considering the *Whitebox Next Gen* design constraints is the point-based architecture. These networks process each point as a single unit, and generate a classification based on the point-specific features.

2.1.1 PointNet

From among a number of point-based classification methods, **PointNet** (Qi et al., 2017) was identified as the strongest overall candidate. PointNet is seen as a watershed approach to LiDAR point cloud classification, having pioneered the point-based approach. It utilizes a series of shared multi-layer perceptrons (MLPs) to infer both point level and global features, then concatenates them together to make a final classification based on the merged feature space. The PointNet framework introduced two key innovations, the **T-Net alignment framework** and the **global feature vector**.

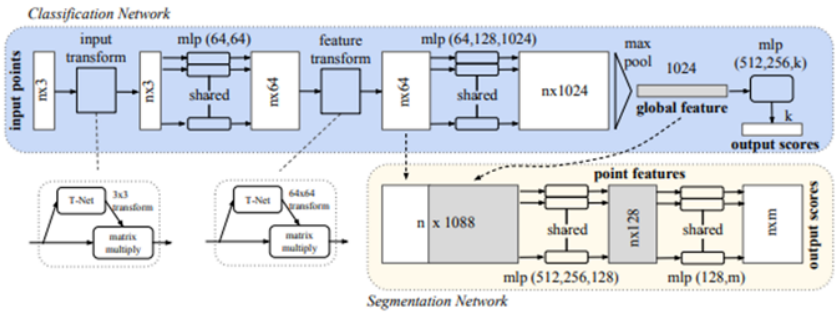

Figure 1: The PointNet architectural diagram. Reprinted from Qi et al. (2017).

T-Net alignment frameworks, which resemble mini-PointNet implementations, solve the key issue of rotational sensitivity within the processing pipeline. Without some solution to this issue, the model would likely learn the most common ”pose” of objects, for example that bridge decks most often run east to west. Then, when encountering a bridge that runs north and south, the system would perform poorly. T-Net alignment frameworks solve this issue by learning a canonical orientation for the data, effectively normalizing the object pose. PointNet makes use of two T-Net alignment frameworks. The first acts on the primary spatial coordinates, producing a 3 x 3 affine transformation matrix that preserves the linearity and parallelism of the features while orienting them to a shared orientation. The second T-Net works on an expanded representation of all point features, predicting a 64 x 64 alignment matrix in high-dimensional space. Because it is challenging to reliably optimize a 64 x 64 matrix without collapsing feature details or creating instability, a regularization term (1) is used to enforce near-orthogonality.

$$L_{\text{reg}} = \left\| I - AA^T \right\|_F^2 \tag{1}$$

This term effectively penalizes the loss function based on how much the product of the transformation matrix A and its transpose A^T deviates from the identity matrix I. This regularization forces the 64 x 64 transformation matrix to preserve the high-dimensional structure and relationships embedded in the features while achieving a canonical orientation.

The global feature vector is created by applying a max pooling operation to the 1024 dimensional feature set, capturing the maximum value from each feature. This is a symmetric function that results

in the same global feature vector regardless of the order of the points, another distinguishing feature of LiDAR point cloud data. The global feature vector is then appended to the 64 dimensional point-level data, creating a multi-scoped 1088 dimensional final feature set that is dimensionally condensed before resulting in a set of class probabilities via the softmax function.

The hallmarks of the PointNet system are highly efficient performance in both semantic segmentation and scene classification via two distinct output pathways, and strong overall classification performance. However, the PointNet++ follow-up architecture identifies PointNet as lacking local context retention (Qi et al., 2017), noting that it ignores the key geometric cues that can be found in the local neighbourhood - beyond the context of the single point but more localized than the global context layer.

2.2 Eigenvalue Feature Augmentation

To provide local context awareness to the PointNet architecture, geometric features derived via Principal Component Analysis (PCA) (Table 1) are appended to each point's attribute record before modeling. Using the *Whitebox Next Gen* `lidar_eigenvalue_features` tool, eigenvalues $(\lambda_1, \lambda_2, \lambda_3)$ are calculated across spatial dimensions within an N-point neighborhood. These eigenvalues yield geometric features such as linearity, planarity, and sphericity, embedding localized structural information directly into the input point features.

2.3 Training and Evaluation Data

A number of LiDAR datasets were evaluated on a variety of criteria, with the Utah Geological Survey (2013) (UGS) and Institut national de l'information géographique et forestière (IGN) (2021) (IGN HD) datasets being used for additional testing in the modeling phase. These two datasets were selected based on their satisfaction of the ASPRS LAS compliance criteria, the presence of key classifications such as water and building within the dataset, and their large overall size which offered plentiful data availability for training and evaluation. Through evaluation of the training performance of each dataset, it became clear that the UGS dataset had data quality issues that

Table 1: Geometric features derived from point neighborhood eigenvalues.

Feature	**Formula**	**Description**
Lambda 1	λ_1	1st eigenvalue – max variance
Lambda 2	λ_2	2nd eigenvalue – orthogonal
Lambda 3	λ_3	3rd eigenvalue – normal to plane
Linearity	$\frac{\lambda_1 - \lambda_2}{\lambda_1}$	Measures linear nature of neighborhood
Planarity	$\frac{\lambda_2 - \lambda_3}{\lambda_1}$	Measures 2D planar nature of neighborhood
Sphericity	$\frac{\lambda_3}{\lambda_1}$	Measure 3D volumetric nature of neighborhood
Omnivariance	$\sqrt[3]{\lambda_1 \times \lambda_2 \times \lambda_3}$	Volumetric size of neighborhood scatter
Curvature	$\frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$	Deviation from flat plane
Eigentropy	$-\sum_{i=1}^{3} p_i \ln(p_i)$ $p_i = \frac{\lambda_i}{\lambda_1+\lambda_2+\lambda_3}$	A measure of structural order in neighborhood points
Slope	$\cos^{-1} \lvert n_z \rvert$	Angle between the local normal vector and the z-axis
Residual	$(p_i - \bar{p}) \cdot v_3$	Distance between a point and its neighborhood plane

could degrade the final model. Because the dataset had not been through a hand validation or human correction stage during classification, and the data had voids and patterns of inconsistent classification (Figure 2). These issues made the UGS dataset an attractive but fundamentally unreliable option.

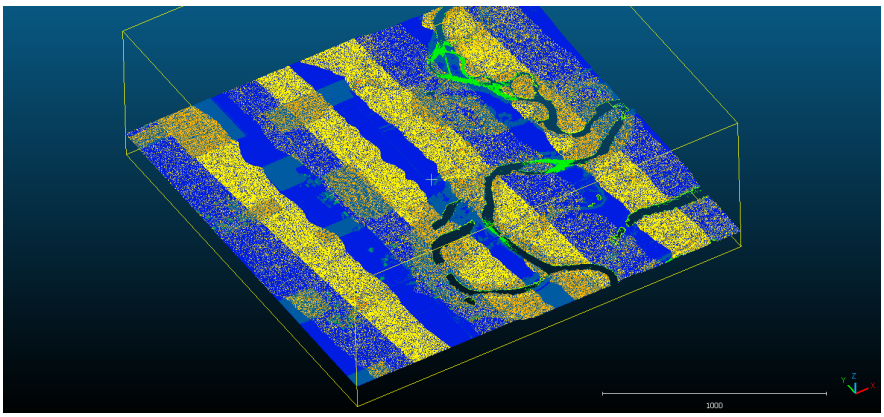

Figure 2: An example of inconsistent classification and data voids in the ot_BearRiver_000024.laz file, taken from the UGS (Utah Geological Survey, 2013) dataset. Captured using CloudCompare.

This left the IGN HD dataset as the primary training and evaluation dataset. Since using the entirety of the dataset would be impossible due to file storage and hardware processing constraints, a two-phase sampling approach was used. In the first phase IGN HD data was manually extracted in alignment with the FRACTAL dataset (Gaydon et al., 2024) - a subsetted and reformatted dataset based on IGN HD and designed for LiDAR point cloud classification system benchmarking and testing. It is worth noting that the FRACTAL dataset was considered for training this model as well, but was rejected due to class misalignment with ASPRS LAS classification standards. By capturing IGN HD data from the FRACTAL project focal areas (Figure 3), it was ensured that the baseline project data was drawn from areas professionally screened for their data diversity and class balance.

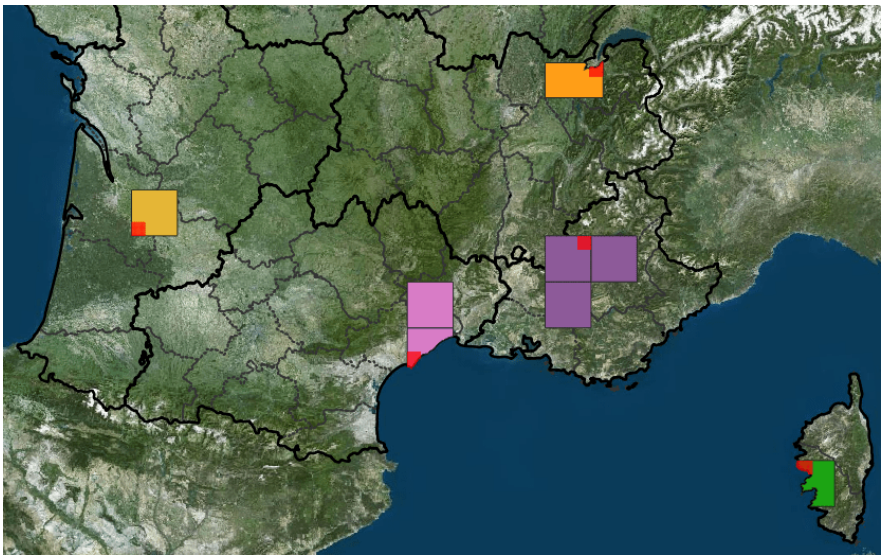

Figure 3: Sampling locations for the FRACTAL dataset. IGN HD data for this project was drawn from the south-central, western-most, and northern-most regions. Reprinted from (Gaydon et al., 2024).

Second-phase sampling was executed via a Python Jupyter notebook program. Key metrics were calculated for each candidate tile within the Phase 1 sample, including a count of total points, the point density, the Shannon entropy of the data in the tile, and the point count for key minority classes like buildings and bridge decks. These metrics were manually assessed and a set of the 12 most diverse tiles from across all 3 Phase 1 sampled areas was identified as the final training sample.

2.4 Exploratory Testing

To better understand the mechanics of the PointNet system and explore the data pre- and post-processing steps necessary for LiDAR point cloud classification, an informal classification pipeline was built using Python and the `Pointnet_Pointnet2_pytorch` GitHub repository (Yan, 2019). This experiment, while brief, was useful for establishing the importance of breaking large files into blocks with an overlapping mechanism and compensating for minority classes in cross-entropy loss calculations.

2.5 AI-Assisted Programming

A key element of success in this project was the use of AI-assisted programming tools and techniques. This approach enabled the use of Rust, despite a lack of deep familiarity with the Rust language and the steep learning curve associated with gaining Rust proficiency. As a result, the project remained on track, with a focus on system coordination, iteration, testing, and refinement rather than the syntactical ins and outs of the Rust language.

As the project progressed, the AI-assisted programming workflow evolved and matured across several tools. A key inflection point in the project was on June 1st 2026, when GitHub Copilot switched from a flat-fee subscription-based billing model to token usage-based accounting. This caused a rapid depletion of the previously allocated budget for AI-assisted programming, and forced a re-evaluation of the AI-assisted programming workflow.

This evaluation was focused on two fronts: AI-assisted programming tools, and the updating the workflow supporting the AI-assisted programming work.

2.5.1 AI-Assisted Programming Tools

Several tools were tested during the transition away from GitHub Copilot, highlighting the importance of critical features like context monitoring and compaction, separate *Plan* and *Act* modes, and tool-use compatibility to enable functionality like online search. **Cline** (Cline Bot Developers,

2024) proved a capable replacement for GitHub Copilot, offering those key features, a variety of models including free models with generous daily token limits, and reasonable pricing.

An array of models were used with the Cline agent, with the bulk of the AI-assisted coding work falling to **Claude Sonnet 5.0** (Anthropic, 2026b). For particularly sticky coding problems **Claude Opus 4.8** (Anthropic, 2026a) was deployed for targeted troubleshooting and deep-dive analysis. For documentation and technical writing purposes, **Deepseek-V4-Flash** (DeepSeek-AI, 2026) was utilized alongside the Claude models to analyze the codebase and draft documentation.

2.5.2 AI-Assisted Programming Workflow

The GitHub Copilot billing changes brought the importance of a refined and meaningful AI-assisted programming workflow into focus. Some key methodological techniques arose from this process.

Detailed Specification This project utilized several layers of detailed project specifications, targeting different scopes and workflows within the project. Defining these specifications consumes a significant portion of the developer's time, as it is critical to ensure they are clear, comprehensive, and coordinated across layers. The first spec defined was the `AGENTS.md` spec; defining the role, standards, expectations, and guardrails for the agent. Building upon that is a global specification file, `PROJECT_SPEC.md` in this case. This spec defines the scope, milestones and expectations of the application and development process. Once those standards are established, specifications for each development stage, or sprint, can be developed. These specs should define the scope of the stage, the expected outcomes, relationship of the stage to the rest of the project, and include a clear definition of done. AI assistance can, and often should, be used in the development and refinement of specifications, but detailed human review and editing are critical since these specs represent the developer's primary interface with the AI assistant.

Context Management Once a development stage is well specced and underway, close attention must be paid to the agent's context window. A context buffer must be compacted **before** it is full to ensure no important details are lost, ideally at 50 to 70% capacity, a process that is often automated

by the agentic harness. This step, while critical, also results in a reduction in context fidelity over time, and it is important for developers to monitor the frequency of compaction steps. Because context fidelity suffers gradual degradation over the lifetime of a particular work session, it is equally important to start a fresh work session between stages. When a new work session is started, the baseline agent context must be rebuilt via a comprehensive review of the project specifications. It can be valuable to have a template initializing prompt that can be used to start a new work session in a clear and standardized way. This prompt might include details on the agent's role, specific files and documentation that need to be integrated into the context, and target task for the work session.

2.6 Modeling Pipeline

The final modeling pipeline, developed purely in Rust in compliance with *Whitebox Next Gen* design standards using AI-assisted coding methods, consists of two primary programs - `wb_lidar_classify` and, with the `training` feature flag enabled, `wb_lidar_train`. These two binaries[1] carry the full functionality of the LiDAR point cloud semantic segmentation modeling pipeline.

2.6.1 The `wb_lidar_classify` tool

The `wb_lidar_classify` program (Figure 4) is the primary compiled package, and does not require any specific feature flags to be enabled at compile time. It is a lightweight and new-dependency free LiDAR classification module that intakes a single `.las`, `.laz`, or `.copc.laz` file, and passes it through a set of configurable preprocessing steps, producing a set of `.feat` files containing the processed input features in a binary-format for each point in the block, and a `blocks.json` file that serves as a manifest of all the blocks created. These blocks are then read by the classification tool, a lightweight implementation of the PointNet mechanism that relies on *ndarray*-based matrix multiplication and related functions for inference calculation - meeting the *Whitebox Next Gen*

[1] A note on documentation: Creating a comprehensive listing of all possible pipeline configuration arguments within this section risks the listing later becoming a stale and confusing reference document. Instead, key arguments will be discussed here, and users should reference the GitHub repository (`https://github.com/JasonIves/lidar_point_cloud_classifier`) for complete and up to date documentation on the available arguments and general framework details.

low-barrier hardware requirements. The inference pass results in an ASPRS LAS v1.4 compliant `.las`, `.laz`, or `.copc.laz` file being output to drive. The format is selectable by the user and does not have to match the input format.

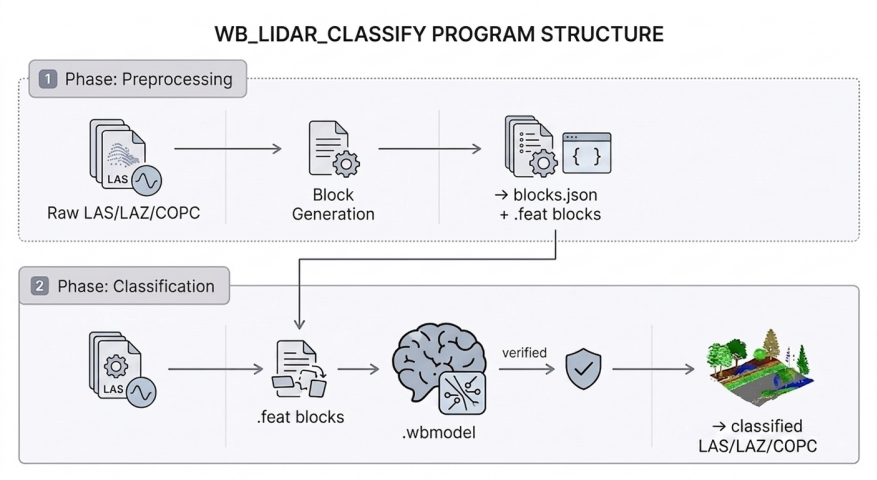

Figure 4: The `wb_lidar_classify` program architecture. Graphic generated using Google Gemini (Google, 2026).

`preprocess` The `preprocess` argument initializes the data preprocessing portion of the `wb_lidar_classify` pipeline. While best practice is to process the input data in the same manner as the model training data, a suite of configuration arguments remain available for user modification. Notable arguments are:

- **`--block-size:`** Defines the single-side length of the blocks created during the tile blocking process.
- **`--target-points:`** The number of points each block will have after under or oversampling.
- **`--block-overlap:`** The amount of overlap between adjacent blocks. Works with `--halo-fraction` to allocate a portion of the `target-points` in the block to overlap representation, ensuring cross-block spatial awareness.
- **`--min-neighbors:`** The number of neighbors included in the principal components analysis (PCA) calculation, for use in eigenvalue feature derivation.

- **`--hag-model`:** Allows users to provide a digital terrain model (DTM) file for use in height above ground (HAG) calculations. If a path is not provided a DTM is generated using the `improved_ground_point_filter` and `lidar_tin_gridding` *Whitebox Next Gen* tools.

`classify` The `classify` argument initializes the classification process, intaking the `.feat` files created during preprocessing. Since the model is pre-trained, the bulk of the available arguments relate to file I/O. One exception worth noting is the `--fusion-radius` argument, which enables cross-block classification voting if `block-overlap` was used in preprocessing. This is a post-classification smoothing mechanism designed to mitigate hard class boundaries between blocks.

2.6.2 The `wb_lidar_train` tool

`wb_lidar_train` is created as a distinct feature-flagged binary (Figure 5) within the project architecture. It is designed to work independently of the `wb_lidar_classify` tool, with a focus on supporting the model development process rather than the forward-facing user experience. This module engages a new Rust dependency, the *Burn* (Tracel AI and Burn Contributors, 2023) machine learning package for GPU-enabled (but not required) model training. The primary output from this workflow is a `.wbmodel` file, which contains a full set of PointNet model weights for later deployment as a pre-trained user-facing model.

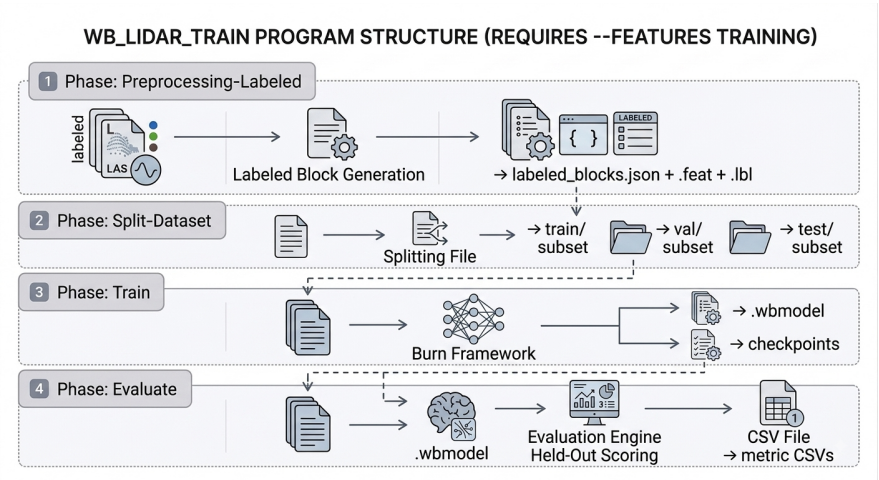

Figure 5: The `wb_lidar_train` program architecture. Graphic generated using Google Gemini (Google, 2026).

`preprocess-labeled` The `preprocess-labeled` argument intentionally shares many functional similarities with the `preprocess` tool discussed in section 2.6.1. The primary functional difference is the creation of a set of `.lbl` files, each containing the class labels for the points represented within the `.feat` file. This separation creates a uniform interface for the `.feat` files, and ensures the class labels remain isolated from the training process. The preprocessing mechanisms detailed in section 2.6.1 remain in place and functionally equivalent. Two additional arguments are required for the model training process:

- **`--label-map:`** The path to a JSON formatted label map, aligning incoming class labels with internal indexing values. This is not a robust reclassification tool, but unwanted classes in the input data can be collapsed to ASPRS LAS Class 1 - *Unassigned* by excluding them from this mapping.
- **`--tile-grid:`** An integer indicating how many different block grouping macro-tiles should be created for each input file. This macro-tiling process minimizes the distribution of spatially autocorrelated blocks across the training, validation, and test splits, reducing data leakage[2].

`split-dataset` The optional `split-dataset` argument creates test, training, and validation datasets based on user-specified split percentages. Macro-tiles as defined in the preprocessing step are distributed across the splits using a stratified algorithmic scoring mechanism (Figure 6) inspired by greedy bin-packing heuristics (Johnson, 1973) that calculates a score for each split based on the target split size and how the per-class proportions within the split compare to the global class distribution.

The lowest scoring split receives the macro-tile in full, and the process continues until all macro-tiles have been allocated. A review and rebalancing pass then ensures that the final splits are as close to the requested proportions as possible. Aside from the standard file I/O and split sizing arguments already discussed, two additionarl arguments implment unique funtionality. The

[2]It should be noted that neighborhood-based eigenvalue calculations persist across tile borders, creating a documented and accepted data leakage scenario at macro-tile borders when adjacent macro-tiles are distributed across splits.

$$C_{\text{size}} = \left(\frac{N_{\text{split}} + N_{\text{tile}}}{N_{\text{grand}}} - f_{\text{target}} \right)^2$$
$$C_{\text{class}} = \sum_c \left(\frac{N_{\text{split},c}}{N_{\text{split}}} - P_{\text{global},c} \right)^2$$
$$C_{\text{total}} = 4.0 \cdot C_{\text{size}} + 1.0 \cdot C_{\text{class}}$$

Figure 6: The formulas for calculating (from top to bottom) the size score C_{size}, the class composition score C_{class}, and the final allocation score C_{total}.

`--no-stratify-classes` argument disables the class-stratified split algorithm, reverting to an every n^{th}, count-based split mechanism. The `--move` flag indicates that the block files should be moved instead of copied. This process is faster and saves total disk space, but is destructive to block source information. Moved block data would need to be reprocessed if a different split configuration were needed.

`train` The `train` argument trains a PointNet model using properly formatted `.feat` and `.lbl` files as the training data. This process outputs one or more `.wbmodel` files, which can be called using the `classify` argument of the `wb_lidar_classify` tool for user-facing classification tasks. It can also output a set of training metrics in `.csv` format, for further review and analysis. This tool leverages a large number of arguments for training configuration and hyperparameter tuning. Some of the most critical arguments are:

- **`--n-classes:`** This integer defines the expected number of output classes. It should align with the label mapping file used during preprocessing.
- **`--batch-size:`** The effective number of blocks accumulated per optimizer step before the AdamW weights are updated.
- **`--forward-batch-size:`** The number of blocks processed in a single pass. These forward batch blocks are a subset of the blocks represented by `block-size`. Gradients are scaled and stored for all forward batches in the batch, then optimizer updates are made based on the

mean gradient of all forward batches. This configuration allows for full control of batch size in a memory-constrained environment.

- **`--use-feature-tnet:`** This flag enables the 64 dimensional feature T-net within the processing architecture, which is one of the most computationally expensive elements in the PointNet architecture. Not enabling this flag should allow for quicker training iteration, but may result in degraded model performance.

- **`--class-weight-beta:`** A factor for increasing the minority class weighting in cross-entropy loss calculations beyond a balanced state. Values approaching 1 yield the strongest preference for minority classes.

- **`--val-split:`** An on-the-fly validation set creation mechanism. Useful for hyperparameter testing and other exploratory phases.

- **`--keep-best-n:`** Indicates how many of the top performing models should be kept, based on validation mean Intersection over Union.

- **`--device:`** Indicates whether the training should be executed using a GPU, CPU, or the processing approach should be auto-detected based on available hardware.

`evaluate` The `evaluate` argument runs a classification pass on the provided model, and evaluates the results against existing class labels. This tool is intended to provide a more unbiased assessment of model performance using unseen test datasets, and outputs only evaluation metrics in `.csv` format - it does not output a classified LiDAR point cloud file. It utilizes the `classify` mechanism, meaning it runs on the CPU only. The available arguments generally mirror those available through `classify` tool, although input of the original `.las`, `.laz`, or `.copc.laz` file is not required. Available arguments include `--metrics-out` and `--confusion-out` for metrics file I/O, and `--n-classes` which runs an on-the-fly cross-check to ensure that the model and dataset share the same expected class count.

3 Results

Once the training pipeline development was stabilized and validated, a candidate model for *Whitebox Next Gen* inclusion was trained. The model showed varied but reasonable performance across classes, with the class-specific Intersection over Union (IoU) ranging between .4135 (Class 17 - Bridge Deck) and .9795 (Class 9 - Water) on unseen test data. Overall semantic segmentation accuracy on the same test data was 84.70%, with a mean Intersection over Union (mIoU) of .6493 across all classes.

Macro-precision, macro-recall and macro-F1 scores across all classes were .7879, .7858, and .7757 respectively. Most intra-class precision and recall values were fairly balanced, with an absolute difference between precision and recall of not more than .16. The exceptions to this parity were the Class 3 - Low Vegetation and Class 17 - Bridge Deck classes.

For the low vegetation class, precision was .8135, while recall was only .5850. This indicates that the model performs somewhat conservatively on low vegetation, only classifying points as low vegetation in a subset of the available instances. One untested hypothesis regarding this discrepancy is that this is due to a blurred or overlaping feature boundary between low vegetation and Class 4 - Medium Vegetation. It is likely that across many types of features these two classes are represented very similarly, and that height above ground is a primary distinguishing feature between them. If the height above ground signal does not create a discriminating difference between the two classes, the model may learn to choose one over the other or defer assigning low confidence points to either class, instead assigning them to a third class like Class 2 - Ground or Class 1 - Unassigned.

Class 17 - Bridge Deck achieved a precision of .4514 and a recall of .8313, representing an overly-aggressive approach to classification. While the model classifies most actual bridge points as such, it also captures a large number of non-bridge points in the process. It stands to reason that this is a result of insufficient representation of the bridge deck class in the training data. Several measures were taken to ensure appropriate representation of minority classes, including targeted tile sampling, class-balance aware dataset splitting, and the use of class-weighted cross-entropy loss in tuning the model weights; but if the defining features of a bridge deck are not represented in the

data with enough strength the model will not be able to learn a full and accurate definition of what is and isn't a bridge deck.

Using additional data in development of future models might serve to define class-specific features more clearly in both of these cases.

3.1 Hyperparameters

Small sample testing revealed a subset of hyperparameters that had a particularly strong influence on model performance (Table 2). These hyperparameters were assessed across a range of values and reasonable and well-performaing values were adopted. It is likely that significant gains in performance could still be achieved through further hyperparameter tuning based on a broader sweep of possible values, and through a multi-factor hyperparameter sweep process. More advanced modeling hardware might be required to explore this high dimensional hyperparameter space efficiently.

Table 2: Key preprocessing and training hyperparameters.

Parameter	Value	Description
PREPROCESSING		
Block size	15	Defines nature of "global" PointNet features
Point density	16	Affects point sampling behavior, available data for modeling, and processing required
Minimum point density	.75	Filters low density blocks
HAG maximum height	50m	Hard ceiling on HAG values
HAG DTM resolution	1m	Tunes the sensitivity of the HAG values to variation in local elevation
Outlier removal	Enabled	Removes points exceeding local mean + buffer
Outlier detection radius	2m	Neighborhood for local mean calculation
TRAINING		
Batch size	32	Effective batch size per optimizer step Accumulates gradients from micro-batches
Forward batch size	8	Input micro-batching Subset of batch size
Learning rate	.001	Starting point of cosine annealing adaptive learning rate
Weight decay	.00005	Weight regularization factor
Class weighting beta	.9999	Factor for over-weighting minority classes

3.2 Block Size Testing

In small sample testing, block size stood out as the single most important factor in model performance. A deeper investigation revealed that, with point density held steady, test set semantic segmentation performance improves when the model is trained with smaller block size (Figure 7). This effect is strong, but not uniform or universal; suggesting there might be value in exploring a mixed or dynamic block size approach.

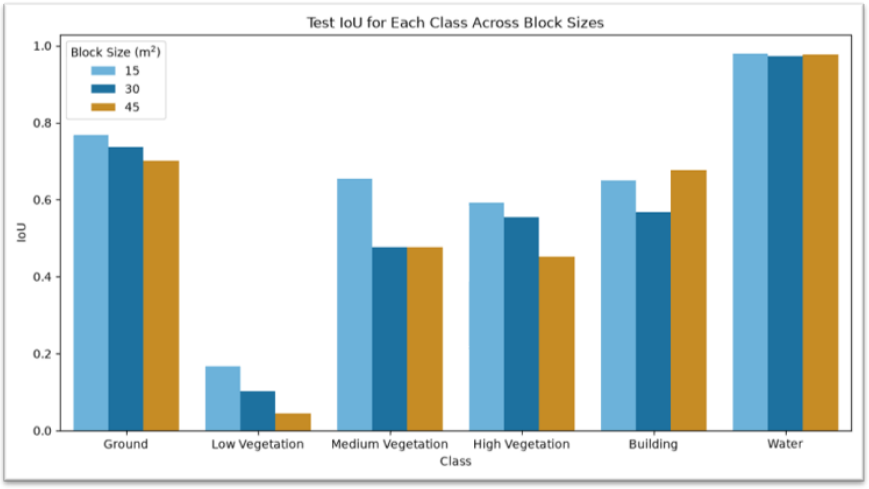

Figure 7: Small sample class Intersection over Union (IoU); using 15, 30, and 45m^2 blocks.

3.3 Validation Performance

The final model was selected from epoch 66 of the training process (Figure 8). The primary selection criterion was that the model had the highest validation mIoU, 0.5642. The selected model was also among those with the lowest class-weighted cross-entropy loss, 0.4228

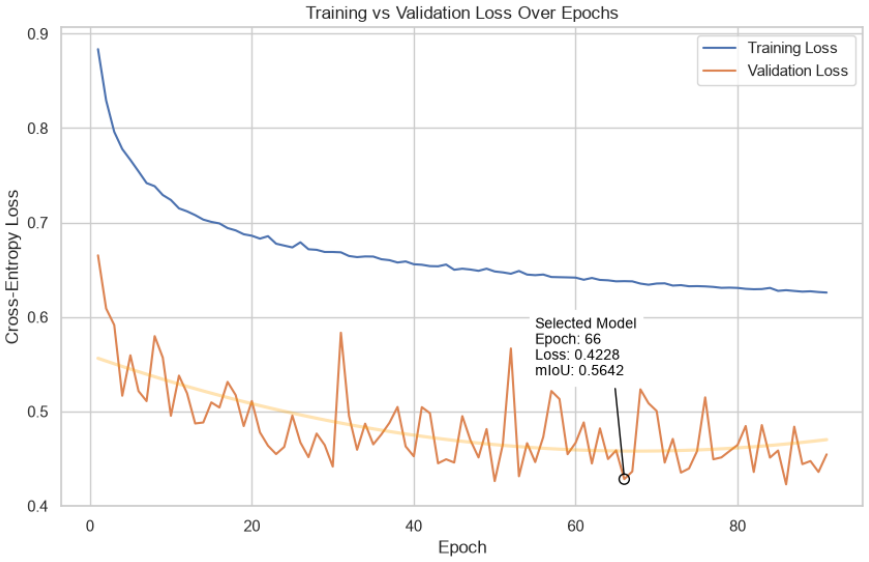

Figure 8: The training and validation loss over epochs for the final training run. The model from epoch 66 was selected due to achieving a maximum mIoU value of 0.5642.

3.4 Per-Class Performance

An analysis of the per-class performance on validation (Figure 9) and test (Figure 10) data sets revealed a range of classification success across classes, with test IoU values ranging between .4135 and .9795. However, both datasets showed similar within-class performance. The model classified Class 9 - Water best across both data sets, with Class 2 - Ground and Class 5 - High Vegetation also being classified well. The model had mixed success identifying classes like 17 - Bridge Deck, 3 - Low Vegetation, and 4 - Medium Vegetation.

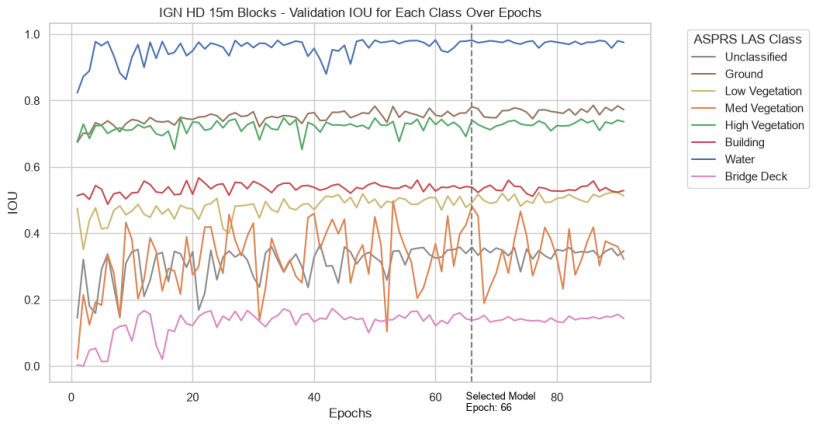

Figure 9: The final model per-class validation set IoU over training epochs.

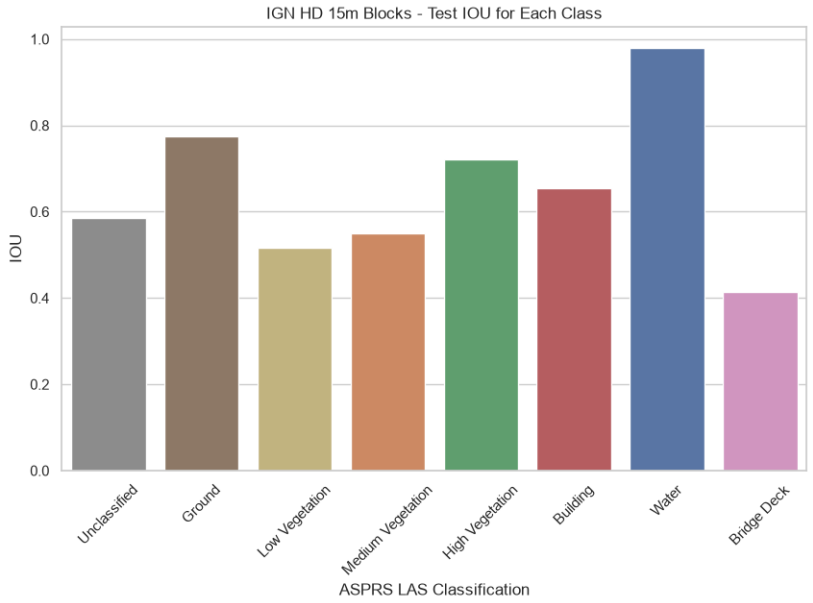

Figure 10: The final model per-class test set IoU.

3.5 Practical Classification

Practical classification testing was done using the UGS (Utah Geological Survey, 2013) and DALES (Varney et al., 2020) datasets. While standard evaluation metrics are not available due to classification misalignment, this sort of functional testing is an important part of evaluating model performance and generalizability (Table 3).

Table 3: Preprocessing and inference statistics for UGS and DALES sample datasets.

Dataset	**Tile Size (m^2)**	**Total Points**	**Density (pts/m^2)**	**Preprocessing Time**	**Processed Blocks**	**Inference Time**	**Inference Rate (blocks/s)**
UGS	2,000	9,703,941	2.43	5m 20s	17,505	19m 35s	14.9
DALES	500	13,886,734	55.50	8m 51s	1,156	2m 18s	8.4

Visual examination of the UGS tile (Figure 11) reveales a pattern of misclassification, showing points along the banks of an unclassified river as bridge deck points. This could be due to their adjacency to an area that is devoid of LiDAR points. Visual examination of the DALES tile (Figure 12) aligns with expectations, showing no obvious classification issues other than an expected level of inaccuracy along block edges.

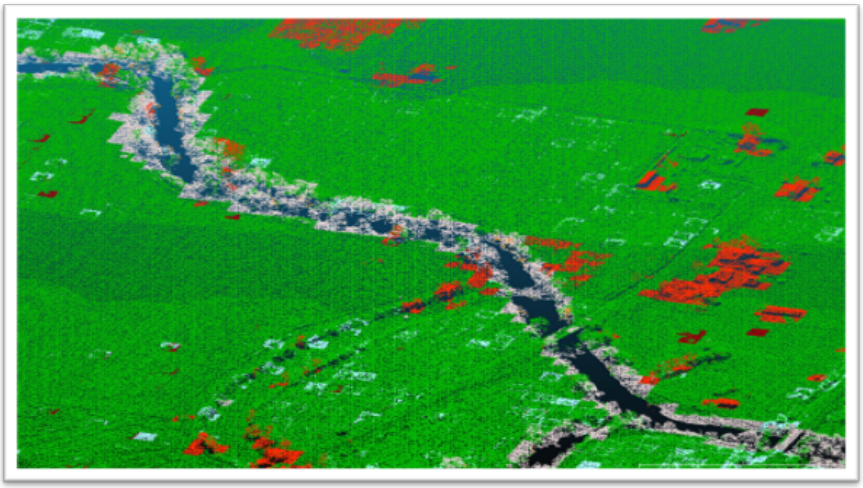

Figure 11: Classification results using a tile from the UGS data set. Green represents ground points, red represents building points, and pink represents bridge points. Captured using CloudCompare.

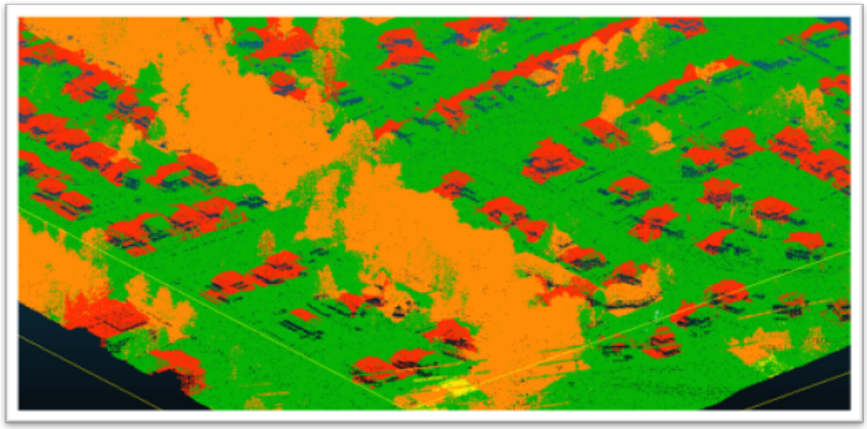

Figure 12: Classification results using a tile from the DALES data set. Green represents ground points, red represents building points, and yellow and orange represent medium and high vegetation respectively. Captured using CloudCompare.

4 Discussion

The core project goal of creating a pre-trained LiDAR point cloud semantic segmentation tool has been accomplished and delivered within the project timeframe. A set of project-specific constraints guided the development process and it is important to examine project success along those dimensions. It is also valuable to look forward to identify areas where the existing model can be improved, as well as examine how the framework can be further developed and refined.

4.1 *Whitebox Next Gen* Design Standards

The *Whitebox Next Gen* development philosophy dictates that the project adhere to five primary constraints

Pure Rust All core tools have been programmed in pure Rust. No C++ or Python wrapped packages have been used in the development. The Rust code makes use of parallelization, multi-threading and data streaming where possible to ensure processing speed is a priority.

Minimal Dependencies The `wb_lidar_classify` program uses no additional dependencies beyond those already used in *Whitebox Next Gen*. It is lightweight and performant, relying on *ndarray* matrix multiplication for its core multi-layer perceptron processing.

The `wb_lidar_train` program makes use of a single new dependency, the *Burn* machine learning package. This decision is in accordance with the "fast first" design principle, enabling highly efficient GPU-based model training. The `wb_lidar_train` module is designed to be developer facing, respecting the lightweight user-facing footprint created by `wb_lidar_classify`.

Cross-Platform The entire LiDAR point cloud classification framework is designed from the ground-up to be functional across all modern computing platforms. This approach has its foundations in the selection of pure Rust as a programming language, which is itself designed to be platform agnostic, and maintains adherence to this approach through its lean dependency footprint and its redundant hardware utilization systems, like its capacity for training on either a GPU or CPU processor.

Standard Hardware The user-facing `wb_lidar_classify` program has been tested on standard consumer-grade hardware, where it has successfully classified millions of LiDAR points in a matter of minutes. While testing on a diverse set of systems would increase confidence in the generalized functionality of the system, this testing provides a strong baseline of evidence.

ASPRS LAS v1.4 / 1.5 Compliant The `wb_lidar_classify` tool generates an ASPRS LAS v1.4 compliant file by utilizing *Whitebox Next Gen's* existing LiDAR reading and writing tools. This preserves the global header geometry from the input file while writing updated point classification data in compliance with LAS v1.4 standards.

4.2 Remaining Issues

While the pre-trained LiDAR point cloud classification model and the related framework are mature and functional, a few areas for possible improvement remain.

Argument Default Updates Since data preprocessing is designed to be a standardized process based on the training parameters of the model, it would make sense to update the argument defaults within the `preprocess` and `preprocess-labeled` scopes to match the preprocessing used to create the pre-trained model. That would ensure users who do not specifically configure the preprocessing pipeline are able to pass data to the model that is parametrically aligned.

Argument Refinement and Pruning Several arguments within the `wb_lidar_classify` and `wb_lidar_train` pipelines have shown minimal benefit in model performance or been functionally replaced by more powerful systems. An audit of existing argument functionality might reveal candidates for streamlining, refinement, or even removal. This process would help keep the framework free of unused bloat and deprecated functionality.

Output Fidelity Two recently discovered gaps remain in the output writing path. First, original Variable Length Records (VLRs), including CRS/GeoTIFF metadata, are not currently copied from the input file to the classified output; only the global header geometry is preserved. Second, `.laz` and `.copc.laz` output paths are redirected to a `.las` ouput path by default, as a patch for an issue that prevents some external tools from reading files that are written out as `.laz` or `.copc.laz`. Both issues are documented and tracked internally, and marked for further investigation.

Cross-Platform and Cross-Hardware Assessment Further testing across an extended range of software and hardware platforms would ensure that the *Whitebox Next Gen* design constraints are rigorously met and increase system availability and adoption.

***Whitebox Next Gen* Compatibility Assessment** *Whitebox Next Gen* is a rigorous and highly performant toolbox. The LiDAR classification tool designed in this project must be duly assessed to ensure that it meets the *Whitebox Next Gen* standards, not only for design, but for performance and compatibility. This process may require framework or model updates, and may or may not result in adoption and integration of the pre-trained LiDAR point cloud classifier into the Whitebox

ecosystem.

4.3 Future Work

In the future, the pipeline and modeling approach developed during this project could be considered building blocks for more ambitious projects.

Model Refinement One important way of building on this project would be development of additional models. The classification tool is capable of supporting multiple models, and there are several avenues available for model improvement.

Hyperparameter Tuning Many of the system hyperparameters have only been tuned to reasonable and logical levels based on small sample focused testing and research. The broader tuning of hyperparameters, especially using multi-parameter cross-validation sweeps, could yield significant model performance gains. Of course, such tuning passes would need to be accompanied by an understanding of the risks of overtuning the hyperparameters to maximize validation dataset performance while sacrificing generalizability.

Data Diversification and Utilization Some issues with the existing model, such as the weakly defined boundary between low and medium vegetation and the imprecise bridge deck classification could be rectified with a larger or more diversified dataset. Dataset diversification would introduce additional variability into the training data, smoothing out any dataset-specific cues that may have been learned while training on single-source data, and introducing additional challenging circumstances for the model to learn from. Utilizing more data to ensure additional instances of rare cases like bridge decks are seen by the model would likely help improve model performance on all classes. There is ample raw LiDAR data available, so the challenge in this would largely lie in the unification of training data labels across datasets and the procurement of hardware capable of handling the increased data throughput in an efficient manner.

Purpose-Built Models While base model improvement is a valuable and achievable goal, the possibility of building purpose-built models for use within this system also exists. Such models might focus on a single class or subset of classes, utilize alternate classification schemas, or be tuned for specific geographic areas or biomes.

Architectural Expansion Other classification systems that fit within the *Whitebox Next Gen* design philosophy could also be added to the ecosystem. Perhaps the modeling pipeline could even be refactored to implement framework modularity, allowing for user selection from a variety of architectures.

The delivery of the `wb_lidar_classify` and `wb_lidar_train` tools along with the accompanying pre-trained semantic segmentation model, meets all core project requirements. More importantly, the development process has mapped out a viable and realistic approach to the integration of deep learning tools into the *Whitebox Next Gen* ecosystem. This project has been deeply educational on a surprising variety of subjects; and has strengthened my understanding of not only data science, machine learning, and geomatics; but of project management, constraint-driven development, and AI-assisted coding. I appreciate the challenging and rewarding opportunity that this project presented, and the stage that it sets for continued growth.

Bibliography

Anthropic (2026a, May). Claude opus 4.8: Technical capabilities and agentic benchmarks. https://www.anthropic.com/news. Accessed: 2026-08-10.

Anthropic (2026b, June). Claude sonnet 5.0: Model overview and capabilities. https://www.anthropic.com/news/claude-sonnet-5. Accessed: 2026-08-10.

Awrangjeb, M. and C. Fraser (2013, 10). Rule-based segmentation of lidar point cloud for automatic extraction of building roof planes. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences II-3/W3*, 1–6.

Cline Bot Developers (2024). Cline: Autonomous ai coding agent. https://cline.bot. Accessed: 2026-08-10.

DeepSeek-AI (2026). Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*.

Gaydon, C., M. Daab, and F. Roche (2024). Fractal: An ultra-large-scale aerial lidar dataset for 3d semantic segmentation of diverse landscapes.

Google (2026). RAG system architectural pipeline. Generative AI image, Gemini. Accessed: 2026-08-10.

Institut national de l'information géographique et forestière (IGN) (2021). Nuages de points LiDAR HD. https://www.data.gouv.fr/datasets/nuages-de-points-lidar-hd. Accessed: 2026-08-08.

Johnson, D. S. (1973). *Near-optimal bin packing algorithms*. Ph. D. thesis, Massachusetts Institute of Technology, Cambridge, MA, USA.

McManamon, P. F. (2019). History of LiDAR. In *LiDAR Technologies and Systems*, pp. 29 – 87. International Society for Optics and Photonics: SPIE.

Qi, C. R., H. Su, K. Mo, and L. J. Guibas (2017). PointNet: Deep learning on point sets for 3D classification and segmentation.

Qi, C. R., L. Yi, H. Su, and L. J. Guibas (2017). PointNet++: Deep hierarchical feature learning on point sets in a metric space.

Sarker, S., P. Sarker, G. Stone, R. Gorman, A. Tavakkoli, G. Bebis, and J. Sattarvand (2024, May). A comprehensive overview of deep learning techniques for 3D point cloud classification and semantic segmentation. *Machine Vision and Applications 35*(4).

Soilán, M., B. Riveiro, J. Balado, and P. Arias (2020). Comparison of heuristic and deep learning-based methods for ground classification from aerial point clouds. *International Journal of Digital Earth 13*(10), 1115–1134.

Tracel AI and Burn Contributors (2023). Burn: A flexible and high-performance deep learning framework in rust. `https://github.com/tracel-ai/burn`.

Utah Geological Survey (2013). Utah Geological Survey LiDAR. Distributed by OpenTopography.

Varney, N., V. K. Asari, and Q. Chen (2020). Dales: A large scale aerial lidar data set for semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 186–187.

Waykar, Y. (2022, 07). Lidar technology: A comprehensive review and future prospects.

Whitebox Geospatial Inc. (2026). Whitebox geospatial: Advanced geoprocessing software.

Yan, X. (2019). Pointnet_pointnet2_pytorch: Pointnet and pointnet++ implemented by pytorch. `https://github.com/yanx27/Pointnet_Pointnet2_pytorch`.